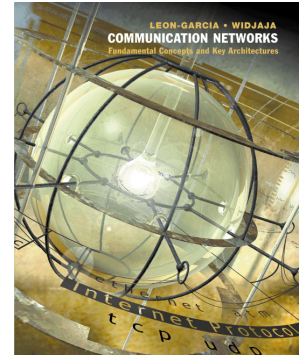
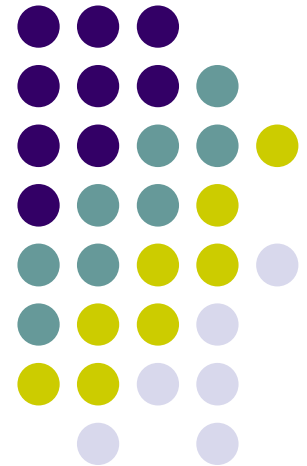


Lecture 2(Chapter 8)

Communication Networks and Services



Transport Layer Protocols: UDP and TCP



Outline



- UDP Protocol
- TCP Reliable Stream Service
- TCP Protocol
- TCP Connection Management
- TCP Flow Control
- TCP Congestion Control

UDP



- Best effort datagram service
- Multiplexing enables sharing of IP datagram service
- Simple transmitter & receiver
 - Connectionless: no handshaking & no connection state
 - Low header overhead
 - No flow control, no error control, no congestion control
 - UDP datagrams can be lost or out-of-order
- Applications
 - multimedia (e.g. RTP)
 - network services (e.g. DNS, RIP, SNMP)

UDP Datagram



0	16	31
Source Port		Destination Port
UDP Length		UDP Checksum
Data		

0-255

- Well-known ports

256-1023

- Less well-known ports

1024-65536

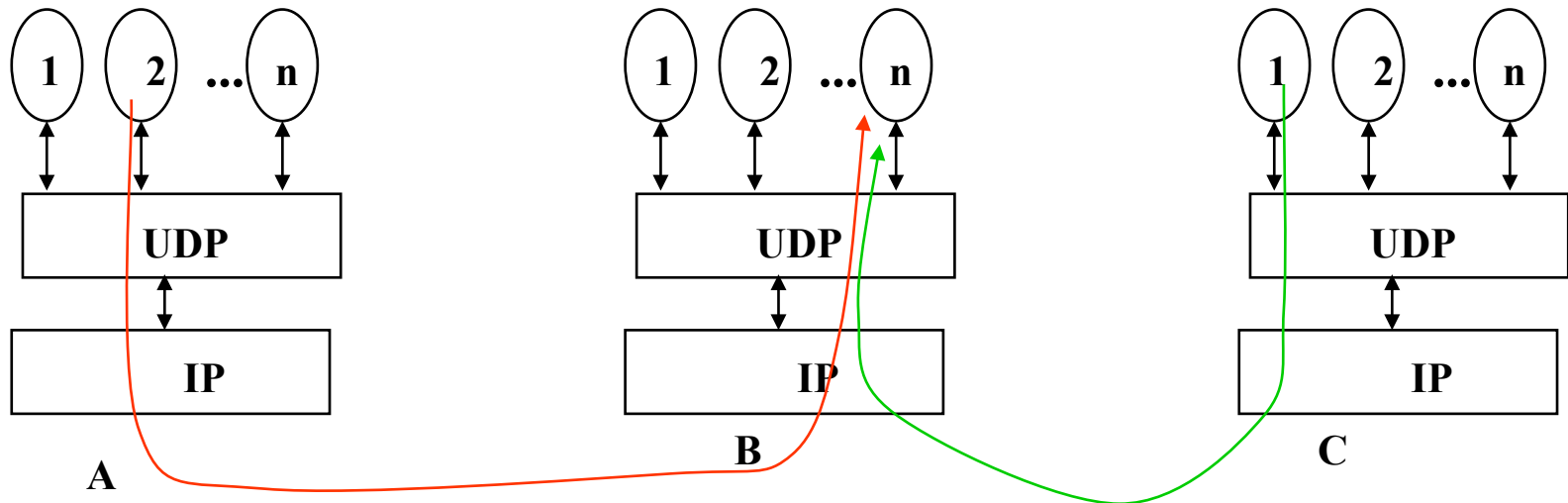
- Ephemeral client ports

- Source and destination port numbers
 - Client ports are ephemeral
 - Server ports are well-known
 - Max number is 65,535
- UDP length
 - Total number of bytes in datagram (including header)
 - $8 \text{ bytes} \leq \text{length} \leq 65,535$
- UDP Checksum
 - Optionally detects errors in UDP datagram

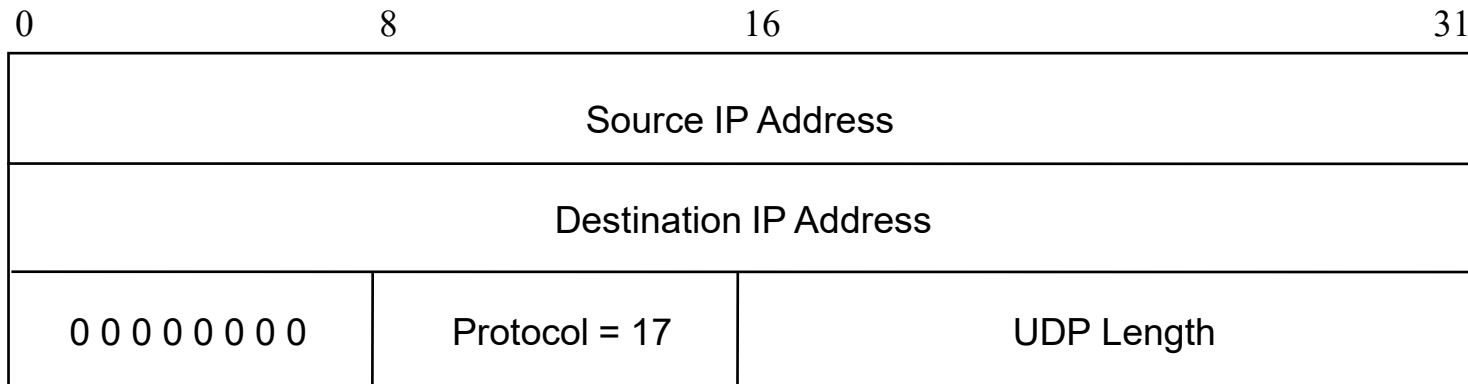
UDP Multiplexing



- All UDP datagrams arriving to IP address B and destination port number n are delivered to the same process
- Source port number is not used in multiplexing



UDP Checksum Calculation



UDP
pseudo-
header

- UDP checksum detects for end-to-end errors
- Covers pseudoheader followed by UDP datagram
- IP addresses included to detect against misdelivery
- IP & UDP checksums set to zero during calculation
- Pad with 1 byte of zeros if UDP length is odd

UDP Receiver Checksum



- UDP receiver recalculates the checksum and silently discards the datagram if errors detected
 - “silently” means no error message is generated
- The use of UDP checksums is optional
- But hosts are required to have checksums enabled

Example



utwebnytimes - Ethereal

File Edit Capture Display Tools Help

No.	Time	Source	Destination	Protocol	Info
20	8.020111	de11_f8:f9:f5	Broadcast	ARP	who has 128.100.11.100?
21	8.577803	128.100.11.13	128.100.100.128	DNS	standard query A www.ny
22	8.707779	128.100.100.128	128.100.11.13	DNS	standard query response

Frame 21 (75 bytes on wire, 75 bytes captured)

- Ethernet II, Src: 00:90:27:96:b8:07, Dst: 00:e0:52:ea:b5:00
- Internet Protocol, Src Addr: 128.100.11.13 (128.100.11.13), Dst Addr: 128.100.100.128 (128.100.100.128)
 - Version: 4
 - Header length: 20 bytes
 - Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00)
 - Total Length: 61
 - Identification: 0x5441
 - Flags: 0x00
 - Fragment offset: 0
 - Time to live: 128
 - Protocol: UDP (0x11)
 - Header checksum: 0x7619 (correct)
 - Source: 128.100.11.13 (128.100.11.13)
 - Destination: 128.100.100.128 (128.100.100.128)
- User Datagram Protocol, Src Port: 1126 (1126), Dst Port: domain (53)
 - Source port: 1126 (1126)
 - Destination port: domain (53)
 - Length: 41
 - Checksum: 0x4983 (correct)
- Domain Name System (query)

0000 00 e0 52 ea b5 00 00 90 27 96 b8 07 08 00 45 00 ..R.... '.....E.
0010 00 3d 54 41 00 00 80 11 76 19 80 64 0b 0d 80 64 .=TA.... v..d...d
0020 64 80 04 66 00 35 00 29 49 83 00 a5 01 00 00 01 d..f.5.) I.....
0030 00 00 00 00 00 00 03 77 77 77 07 6e 79 74 69 6dw ww.nytim
0040 65 73 03 63 6f 6d 00 00 01 00 01 es.com.. ...

Filter: [] [✓] Reset Apply File: utwebnytimes

Outline



- UDP Protocol
- TCP Reliable Stream Service
- TCP Protocol
- TCP Connection Management
- TCP Congestion Control

TCP

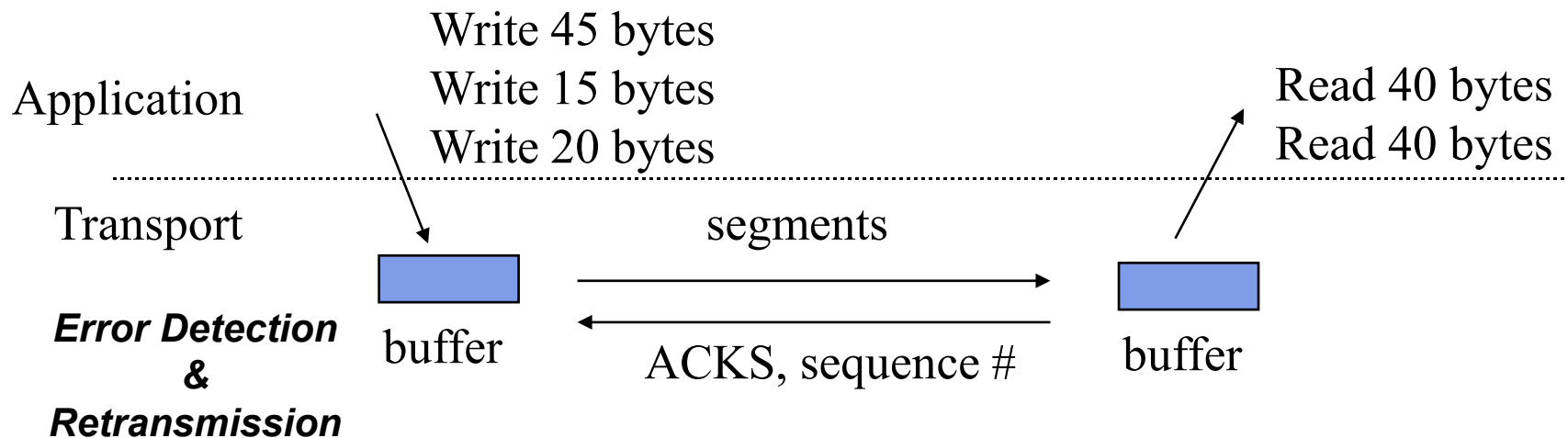


- Reliable byte-stream service
- More complex transmitter & receiver
 - Connection-oriented: full-duplex unicast connection between client & server processes
 - Connection setup, connection state, connection release
 - Higher header overhead
 - Error control, flow control, and congestion control
 - Higher delay than UDP
- Most applications use TCP
 - HTTP, SMTP, FTP, TELNET, POP3, ...

Reliable Byte-Stream Service



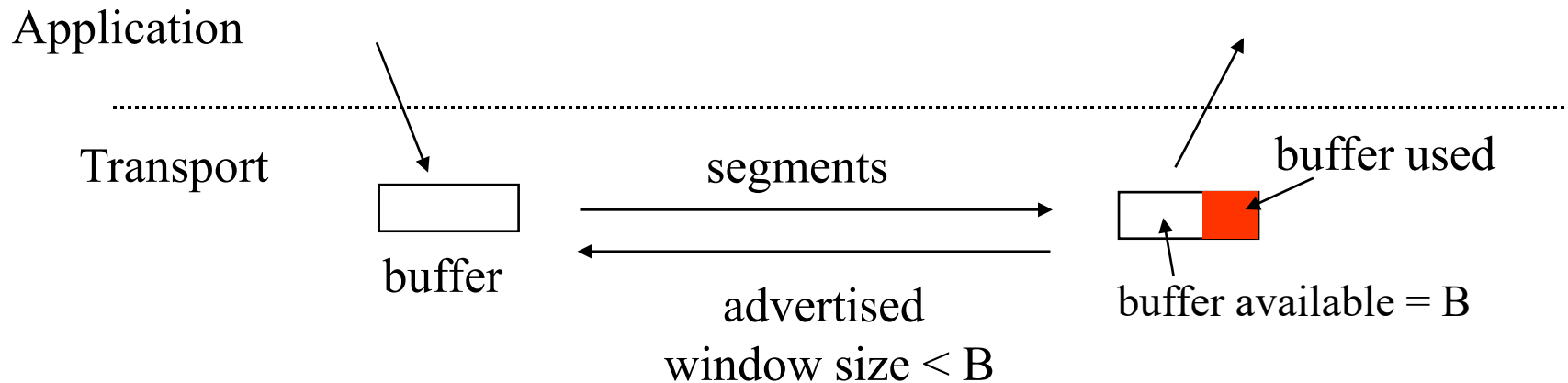
- Stream Data Transfer
 - transfers a contiguous stream of bytes across the network, with no indication of boundaries
 - groups bytes into segments
 - transmits segments as convenient (Push function defined)
- Reliability
 - error control mechanism to deal with IP transfer impairments





Flow Control

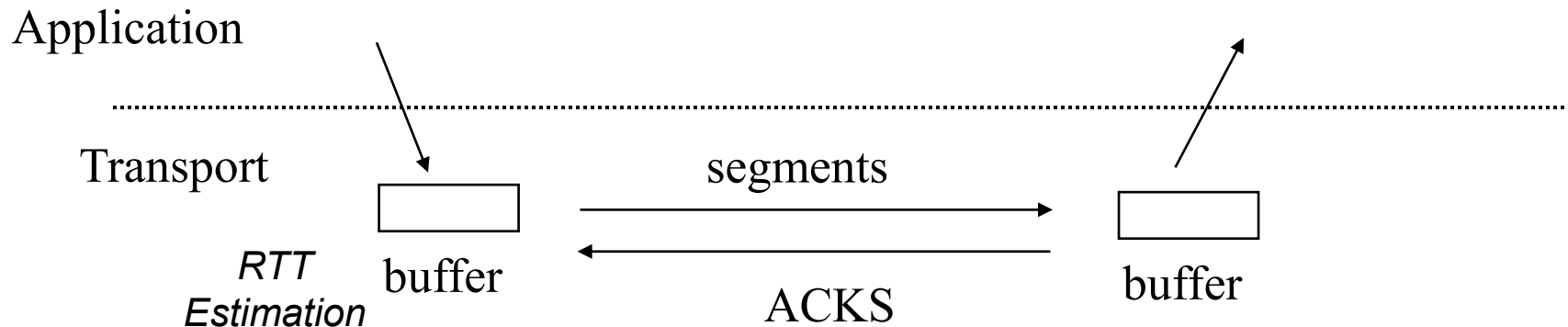
- Buffer limitations & speed mismatch can result in loss of data that arrives at destination
- Receiver controls rate at which sender transmits to prevent buffer overflow



Congestion Control



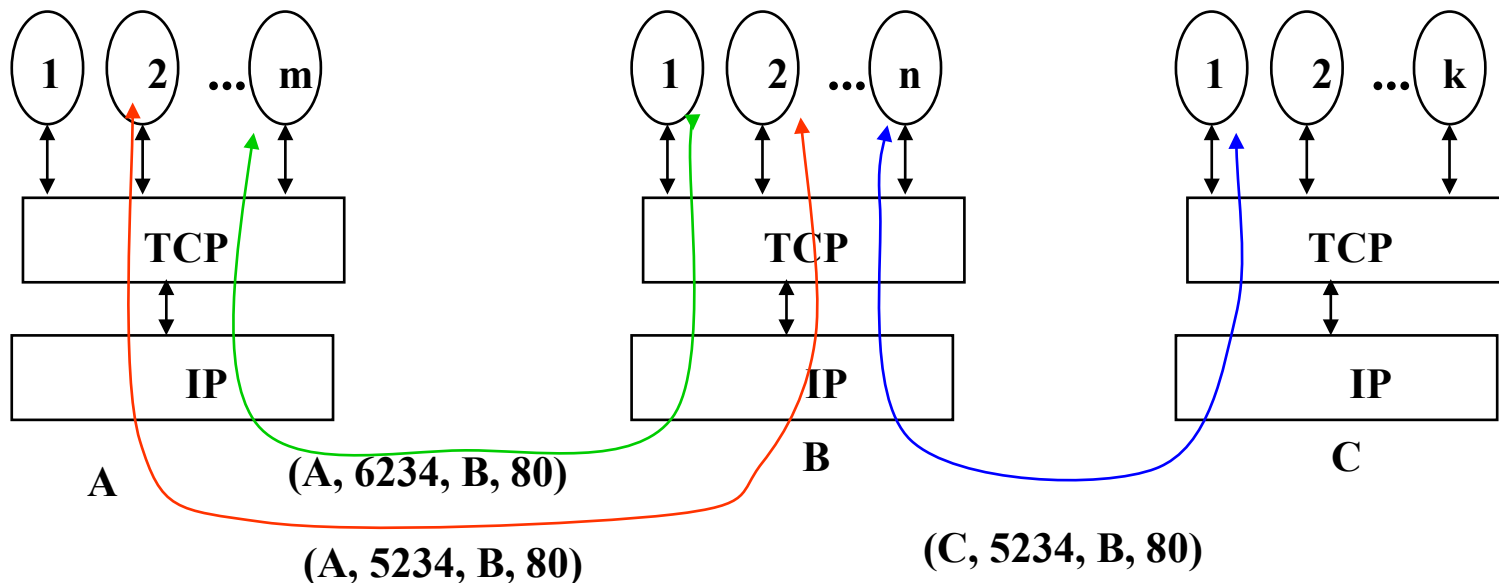
- Available bandwidth to destination varies with activity of other users
- Transmitter dynamically adjusts transmission rate according to network congestion as indicated by RTT (round trip time) & ACKs
- Elastic utilization of network bandwidth





TCP Multiplexing

- A *TCP connection* is specified by a *4-tuple*
 - (source IP address, source port, destination IP address, destination port)
- TCP allows multiplexing of multiple connections between end systems to support multiple applications simultaneously
- Arriving segment directed according to connection 4-tuple

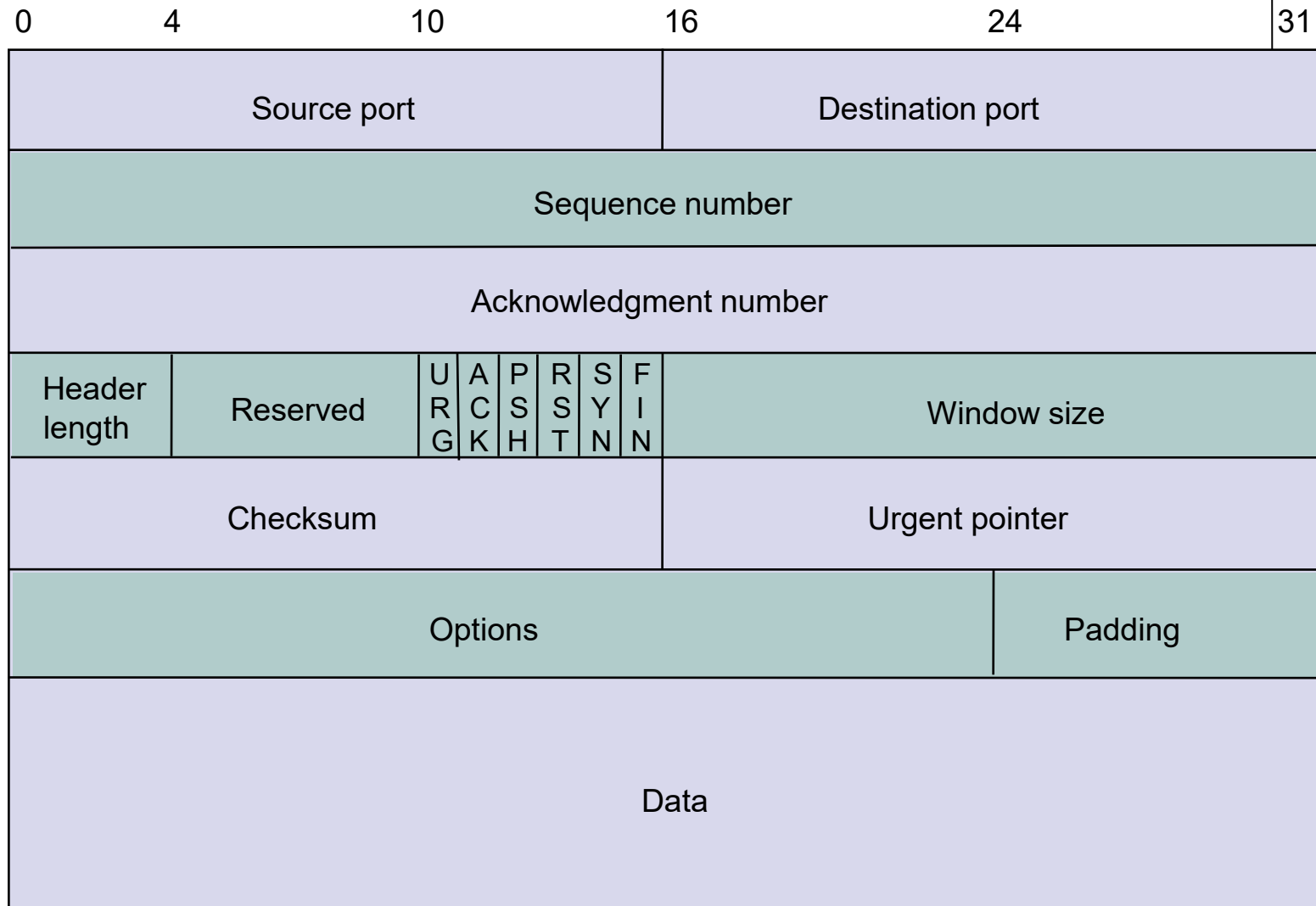




Outline

- UDP Protocol
- TCP Reliable Stream Service
- TCP Protocol
- TCP Connection Management
- TCP Congestion Control

TCP Segment Format



- Each TCP segment has header of 20 or more bytes + 0 or more bytes of data

TCP Header



Port Numbers

- A socket identifies a connection endpoint
 - IP address + port
- A connection specified by a *socket pair*
- Well-known ports
 - FTP 20
 - Telnet 23
 - DNS 53
 - HTTP 80

Sequence Number

- Byte count
- First byte in segment
- 32 bits long
- $0 \leq SN \leq 2^{32}-1$
- Initial sequence number selected during connection setup

TCP Header



Acknowledgement Number

- SN of next byte expected by receiver
- Acknowledges that all prior bytes in stream have been received correctly
- Valid if ACK flag is set

Header length

- 4 bits
- Length of header in multiples of 32-bit words
- Minimum header length is 20 bytes
- Maximum header length is 60 bytes

TCP Header



Reserved

- 6 bits

Control

- 6 bits
- URG: urgent pointer flag
 - Urgent message end = $SN + \text{urgent pointer}$
- ACK: ACK packet flag
- PSH: override TCP buffering
- RST: reset connection
 - Upon receipt of RST, connection is terminated and application layer notified
- SYN: establish connection
- FIN: close connection

TCP Header



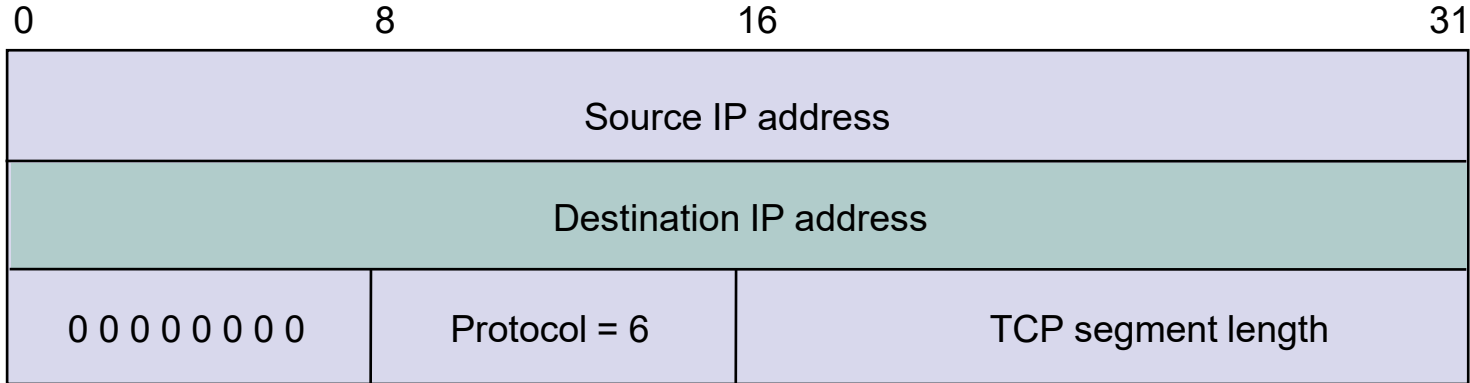
Window Size

- 16 bits to advertise window size
- Used for flow control
- Sender will accept bytes with SN from ACK to ACK + window
- Maximum window size is 65535 bytes

TCP Checksum

- Internet checksum method
- TCP pseudoheader + TCP segment

TCP Checksum Calculation



TCP
pseudo-
header

- TCP error detection uses same procedure as UDP

TCP Header



Options

- Variable length
- NOP (No Operation) option is used to pad TCP header to multiple of 32 bits
- Time stamp option is used for round trip measurements

Options

- Maximum Segment Size (MSS) option specifies largest segment a receiver wants to receive
- Window Scale option increases TCP window from 16 to 32 bits

Outline

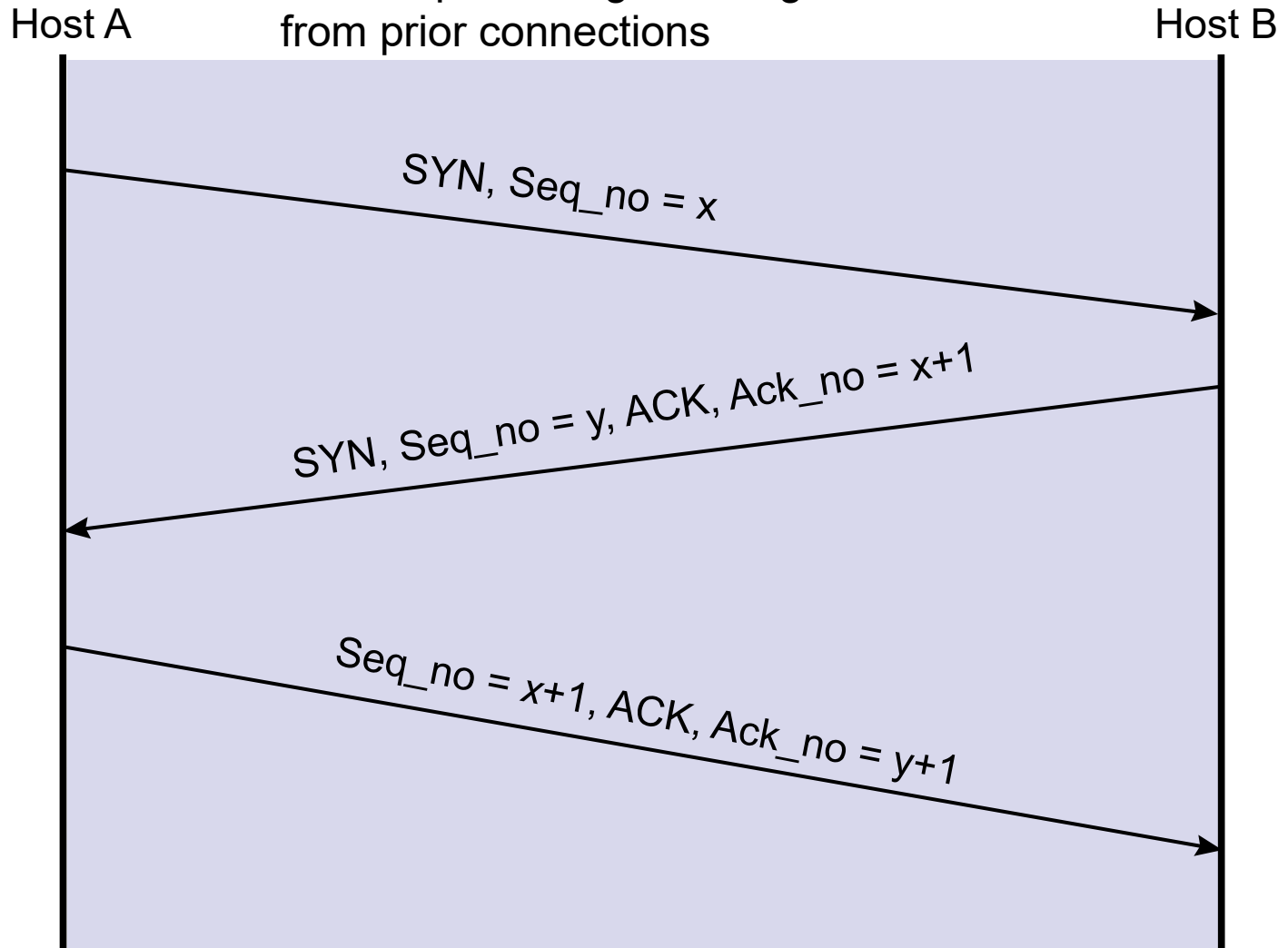


- UDP Protocol
- TCP Reliable Stream Service
- TCP Protocol
- TCP Connection Management
- TCP Congestion Control

TCP Connection Establishment



- “Three-way Handshake”
- ISN’s protect against segments from prior connections





Maximum Segment Size

- Maximum Segment Size
 - largest block of data that TCP sends to other end
- Each end can announce its MSS during connection establishment
- Default is 576 bytes including 20 bytes for IP header and 20 bytes for TCP header
- Ethernet implies MSS of 1460 bytes
- IEEE 802.3 implies 1452

Near End: Connection Request



utwebnytimes - Ethereal

File Edit Capture Display Tools Help

No.	Time	Source	Destination	Protocol	Info
22	8.707779	128.100.100.128	128.100.11.13	DNS	Standard query response A 64.15.247.200 A 64.15.247.200
23	8.709327	128.100.11.13	64.15.247.200	TCP	1127 > http [SYN] Seq=3638689752 Ack=0 win=0
24	8.746089	64.15.247.200	128.100.11.13	TCP	http > 1127 [SYN, ACK] Seq=1396200325 Ack=3638689752 Win=65535 Len=0
25	8.746123	128.100.11.13	64.15.247.200	TCP	1127 > http [ACK] Seq=3638689753 Ack=1396200325 Win=65535 Len=0
26	8.746491	128.100.11.13	64.15.247.200	HTTP	GET / HTTP/1.1
27	8.783242	64.15.247.200	128.100.11.13	TCP	http > 1127 [ACK] Seq=1396200326 Ack=3638689753 Win=65535 Len=0
28	8.814479	64.15.247.200	128.100.11.13	HTTP	HTTP/1.1 200 OK
29	8.814526	64.15.247.200	128.100.11.13	HTTP	Continuation

Frame 23 (62 bytes on wire (92 bytes captured))

Ethernet II, Src: 00:90:27:96:b8:07, Dst: 00:e0:52:ea:b5:00

Internet Protocol, Src Addr: 128.100.11.13 (128.100.11.13), Dst Addr: 64.15.247.200 (64.15.247.200)

Transmission Control Protocol, Src Port: 1127 (1127), Dst Port: http (80), Seq: 3638689752, Ack: 0, Len: 0

Source port: 1127 (1127)

Destination port: http (80)

Sequence number: 3638689752

Header length: 28 bytes

Flags: 0x0002 (SYN)

0... = Congestion window Reduced (CWR): Not set

.0.. = ECN-Echo: Not set

..0. = Urgent: Not set

...0 = Acknowledgment: Not set

.... 0... = Push: Not set

.... .0.. = Reset: Not set

.... ..1. = Syn: Set

.... ...0 = Fin: Not set

Window size: 16384

Checksum: 0xa2e4 (correct)

Options: (8 bytes)

Maximum segment size: 1332 bytes

NOP

NOP

SACK permitted

0000 00 e0 52 ea b5 00 00 90 27 96 b8 07 08 00 45 00 ..R.....E.

0010 00 30 54 42 40 00 80 06 e3 3c 80 64 0b 0d 40 0f .0TB@...<.d..@.

0020 f7 c8 04 67 00 50 d8 e1 ff d8 00 00 00 00 70 02 ...g.P.....p.

0030 40 00 a2 e4 00 00 02 04 05 34 01 01 04 02 @.....4....

Filter: / Reset Apply File: utwebnytimes

Far End: Ack and Request



utwebnytimes - Ethereal

File Edit Capture Display Tools Help

No.	Time	Source	Destination	Protocol	Info
22	8.707779	128.100.100.128	128.100.11.13	DNS	Standard query response A 64.15.247.200 A 64.15.247.200
23	8.709327	128.100.11.13	64.15.247.200	TCP	1127 > http [SYN] Seq=3638689752 Ack=0 win=0
24	8.746089	64.15.247.200	128.100.11.13	TCP	http > 1127 [SYN, ACK] Seq=1396200325 Ack=3638689753
25	8.746123	128.100.11.13	64.15.247.200	TCP	1127 > http [ACK] Seq=3638689753 Ack=1396200325
26	8.746491	128.100.11.13	64.15.247.200	HTTP	GET / HTTP/1.1
27	8.783242	64.15.247.200	128.100.11.13	TCP	http > 1127 [ACK] Seq=1396200326 Ack=3638689753
28	8.814479	64.15.247.200	128.100.11.13	HTTP	HTTP/1.1 200 OK
29	8.814526	64.15.247.200	128.100.11.13	HTTP	Continuation

Frame 24 (82 bytes on wire (66 bytes captured) on interface 0:08:00:27:96:b8)

Ethernet II, Src: 00:e0:52:ea:b5:00, Dst: 00:90:27:96:b8:07

Internet Protocol, Src Addr: 64.15.247.200 (64.15.247.200), Dst Addr: 128.100.11.13 (128.100.11.13)

Transmission Control Protocol, Src Port: http (80), Dst Port: 1127 (1127), Seq: 1396200325, Ack: 3638689753, Source port: http (80), Destination port: 1127 (1127), Sequence number: 1396200325, Acknowledgement number: 3638689753, Header length: 28 bytes

Flags: 0x0012 (SYN, ACK)

- 0... = Congestion window Reduced (CWR): Not set
- .0... = ECN-Echo: Not set
- ..0. = Urgent: Not set
- ...1 = Acknowledgment: Set
- 0... = Push: Not set
-0.. = Reset: Not set
-1. = Syn: set
-0 = Fin: Not set

Window size: 1460

Checksum: 0x35e2 (correct)

Options: (8 bytes)

- NOP
- NOP
- SACK permitted
- Maximum segment size: 1460 bytes

0000 00 90 27 96 b8 07 00 e0 52 ea b5 00 08 00 45 00 .. . R....E.

0010 00 30 b3 91 40 00 ed 06 16 ed 40 0f f7 c8 80 64 .0..@... ..@....d

0020 0b 0d 00 50 04 67 53 38 53 85 d8 e1 ff d9 70 12 ...P.gs8 S.....p.

0030 05 b4 35 e2 00 00 01 01 04 02 02 04 05 b4 ..5.....

Filter: Reset Apply File: utwebnytimes

Near End: Ack



utwebnytimes - Ethereal

File Edit Capture Display Tools Help

No.	Time	Source	Destination	Protocol	Info
22	8.707779	128.100.100.128	128.100.11.13	DNS	Standard query response A 64.15.247.200 A 64.15.247.200
23	8.709327	128.100.11.13	64.15.247.200	TCP	1127 > http [SYN] Seq=3638689752 Ack=0 win=0
24	8.746089	64.15.247.200	128.100.11.13	TCP	http > 1127 [SYN, ACK] Seq=1396200325 Ack=3638689753
25	8.746123	128.100.11.13	64.15.247.200	TCP	1127 > http [ACK] Seq=3638689753 Ack=1396200326
26	8.746491	128.100.11.13	64.15.247.200	HTTP	GET / HTTP/1.1
27	8.783242	64.15.247.200	128.100.11.13	TCP	http > 1127 [ACK] Seq=1396200326 Ack=3638689753
28	8.814479	64.15.247.200	128.100.11.13	HTTP	HTTP/1.1 200 OK
29	8.814526	64.15.247.200	128.100.11.13	HTTP	Continuation

Frame 25 (54 bytes on wire, 54 bytes captured)

Ethernet II, Src: 00:90:27:96:b8:07, Dst: 00:e0:52:ea:b5:00

Internet Protocol, Src Addr: 128.100.11.13 (128.100.11.13), Dst Addr: 64.15.247.200 (64.15.247.200)

Transmission Control Protocol, Src Port: 1127 (1127), Dst Port: http (80), Seq: 3638689753, Ack: 1396200326, source port: 1127 (1127), destination port: http (80), Sequence number: 3638689753, Acknowledgement number: 1396200326, Header length: 20 bytes

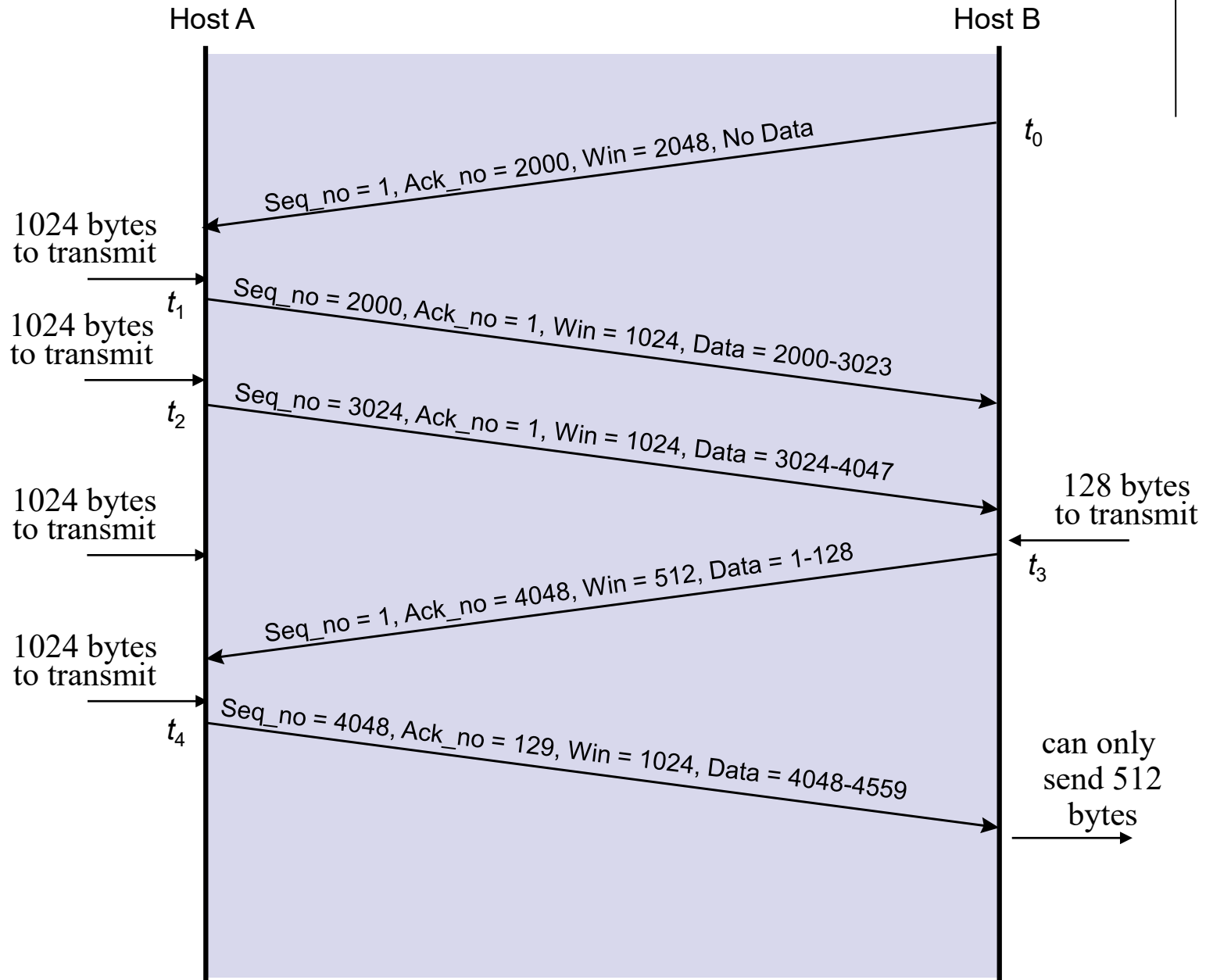
Flags: 0x0010 (ACK)

0... = Congestion window Reduced (CWR): Not set
.0.. = ECN-Echo: Not set
..0. = Urgent: Not set
...1 = Acknowledgment: Set
.... 0... = Push: Not set
.... .0.. = Reset: Not set
.... ..0. = Syn: Not set
.... ...0 = Fin: Not set
window size: 17316
checksum: 0x24b6 (correct)

0000 00 e0 52 ea b5 00 00 90 27 96 b8 07 08 00 45 00 ..R.....'.....E.
0010 00 28 54 44 40 00 80 06 e3 42 80 64 0b 0d 40 0f ..(TD@... .B.d..@.
0020 f7 c8 04 67 00 50 d8 e1 ff d9 53 38 53 86 50 10 ...g.P... .sS.P.
0030 43 a4 24 b6 00 00 C.\$...

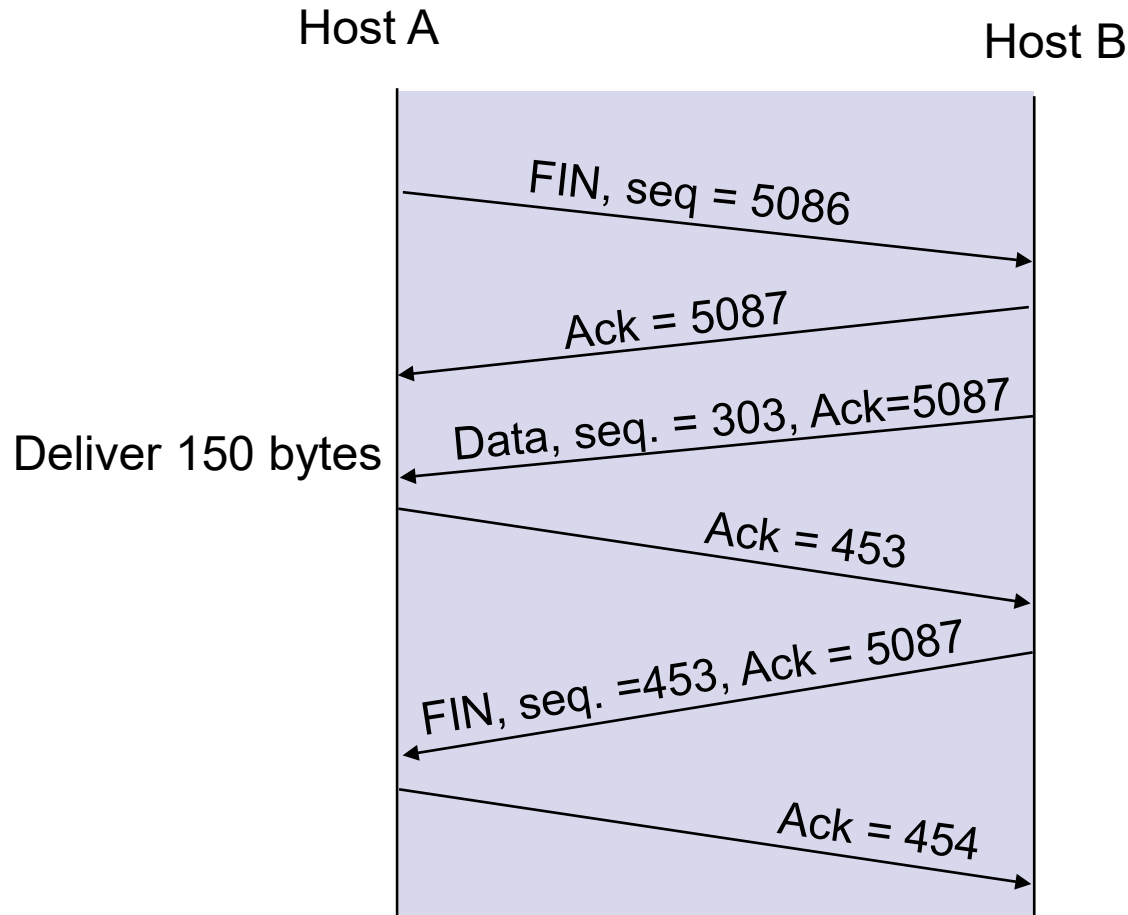
Filter: / Reset Apply File: utwebnytimes

TCP Window Flow Control



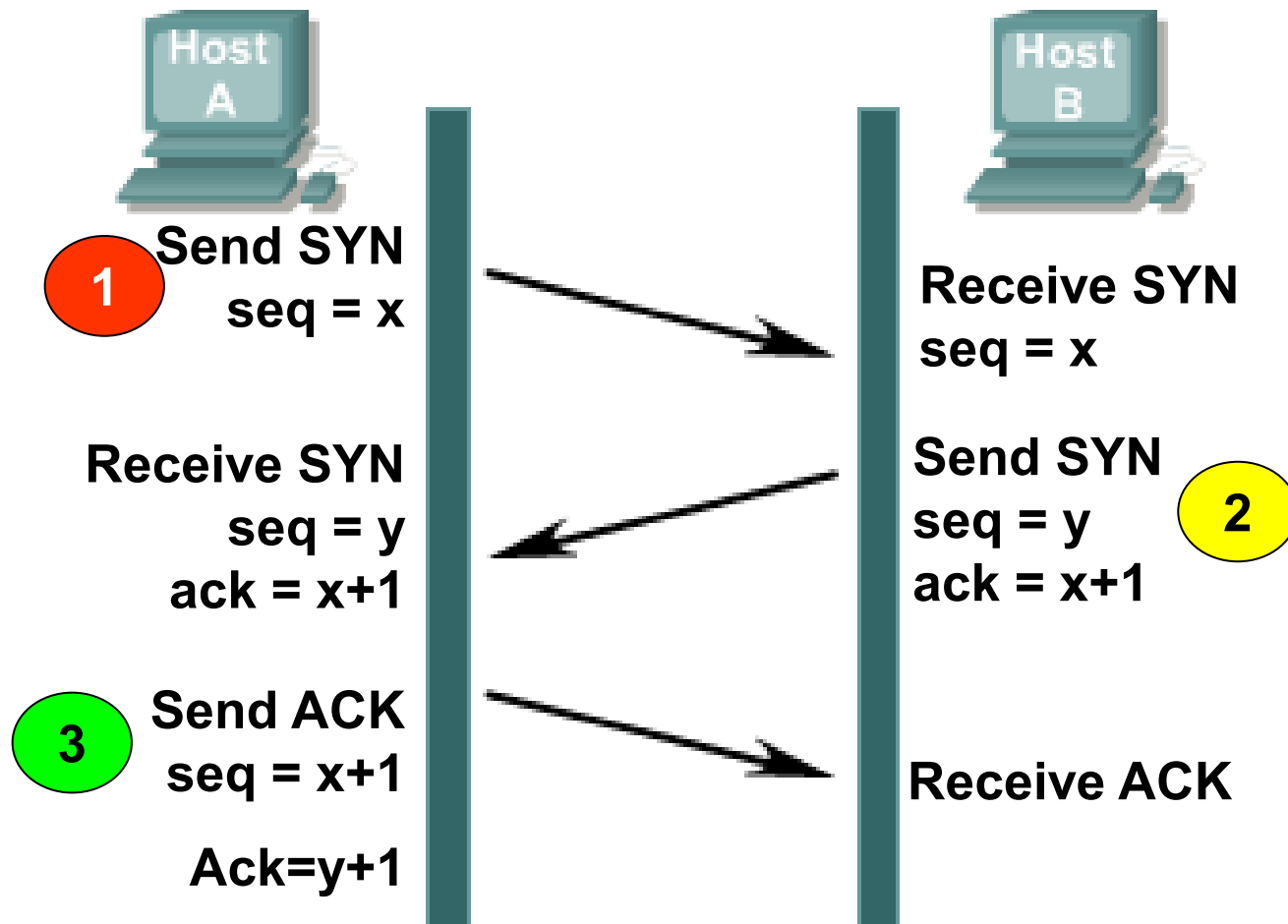
TCP Connection Closing

“Graceful Close”

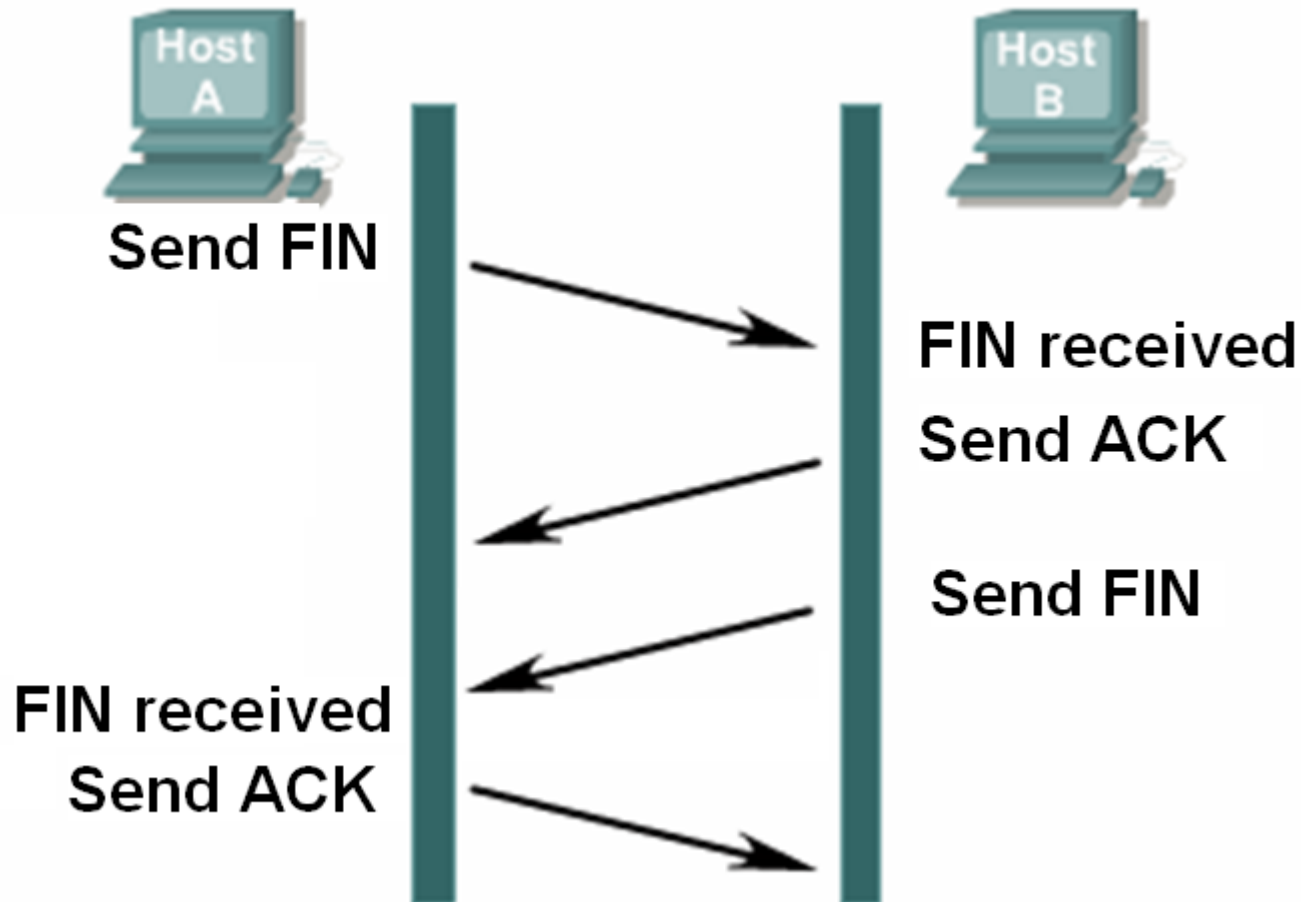




3-Way handshake – Initiating a Session



Terminating a Session



Outline

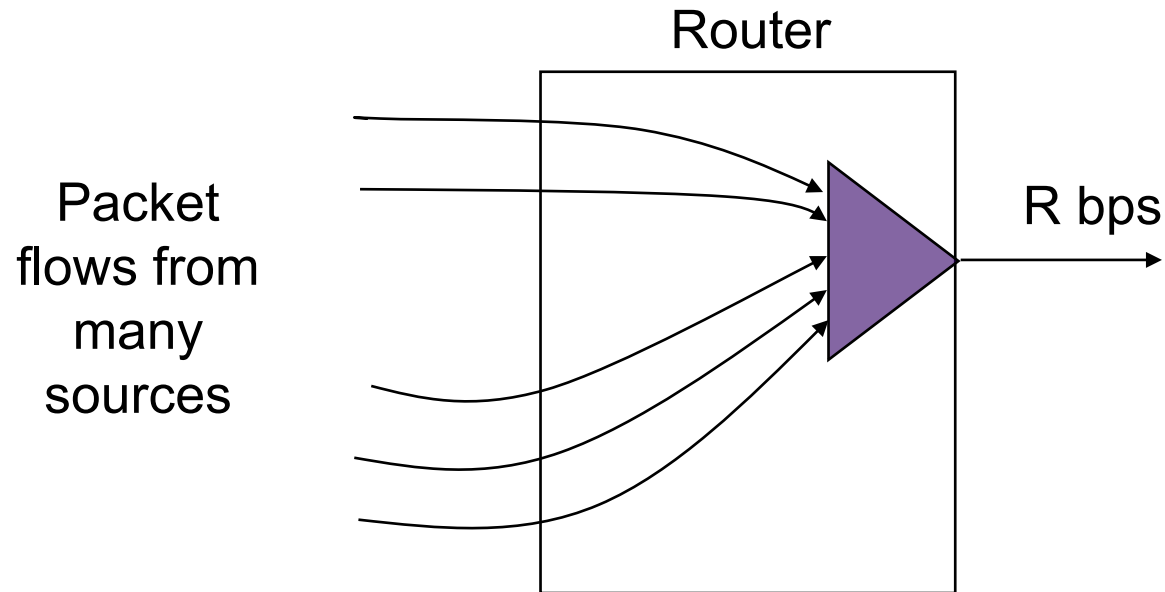


- UDP Protocol
- TCP Reliable Stream Service
- TCP Protocol
- TCP Connection Management
- TCP Congestion Control

TCP Congestion Control



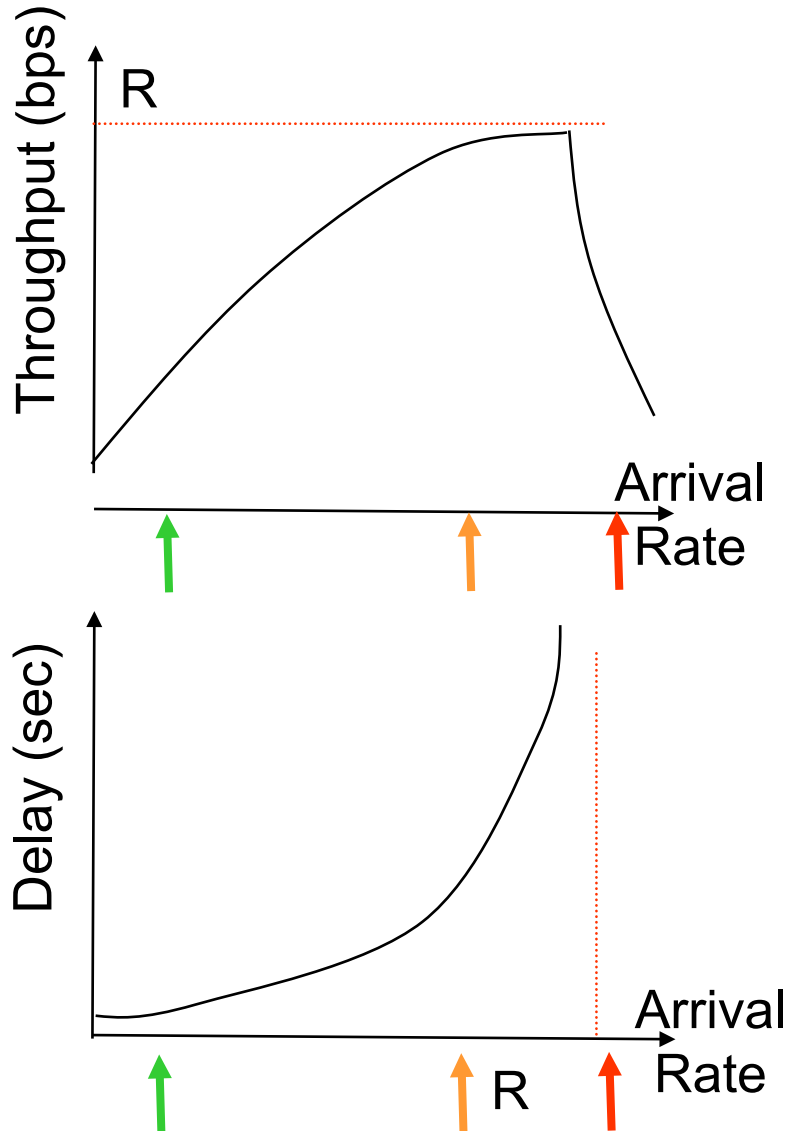
- *Advertised window* size is used to ensure that receiver's buffer will not overflow
- However, buffers at intermediate routers between source and destination may overflow



- Congestion occurs when total arrival rate from all packet flows exceeds R over a sustained period of time
- Buffers at multiplexer will fill and packets will be lost



Phases of Congestion Behavior



1. Light traffic

- Arrival Rate $\ll R$
- Low delay
- Can accommodate more

2. Knee (congestion onset)

- Arrival rate approaches R
- Delay increases rapidly
- Throughput begins to saturate

3. Congestion collapse

- Arrival rate $> R$
- Large delays, packet loss
- Useful application throughput drops

Congestion Window

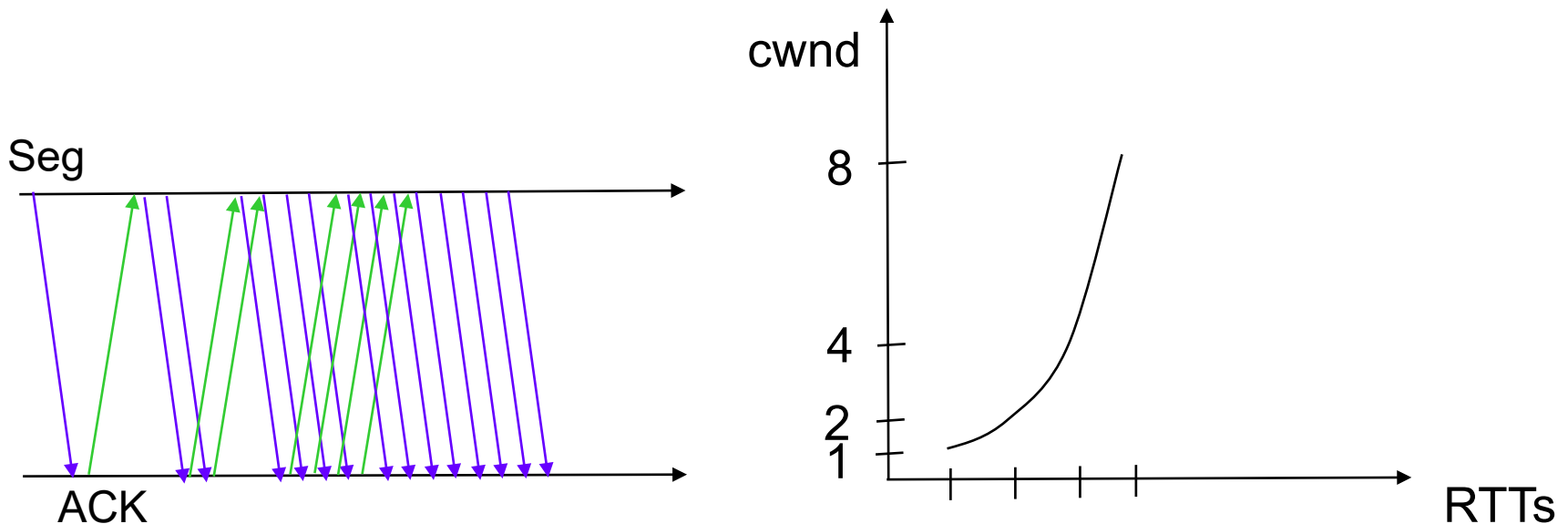


- How does the TCP congestion algorithm change congestion window dynamically according to the most up-to-date state of the network?
- At light traffic: each segment is ACKed quickly
 - Increase cwnd aggressively
- At knee: segment ACKs arrive, but more slowly
 - Slow down increase in cwnd
- At congestion: segments encounter large delays (so retransmission timeouts occur); segments are dropped in router buffers (resulting in duplicate ACKs)
 - Reduce transmission rate, then probe again

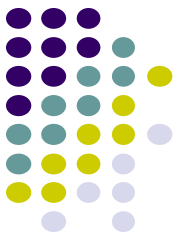
TCP Congestion Control: Slow Start



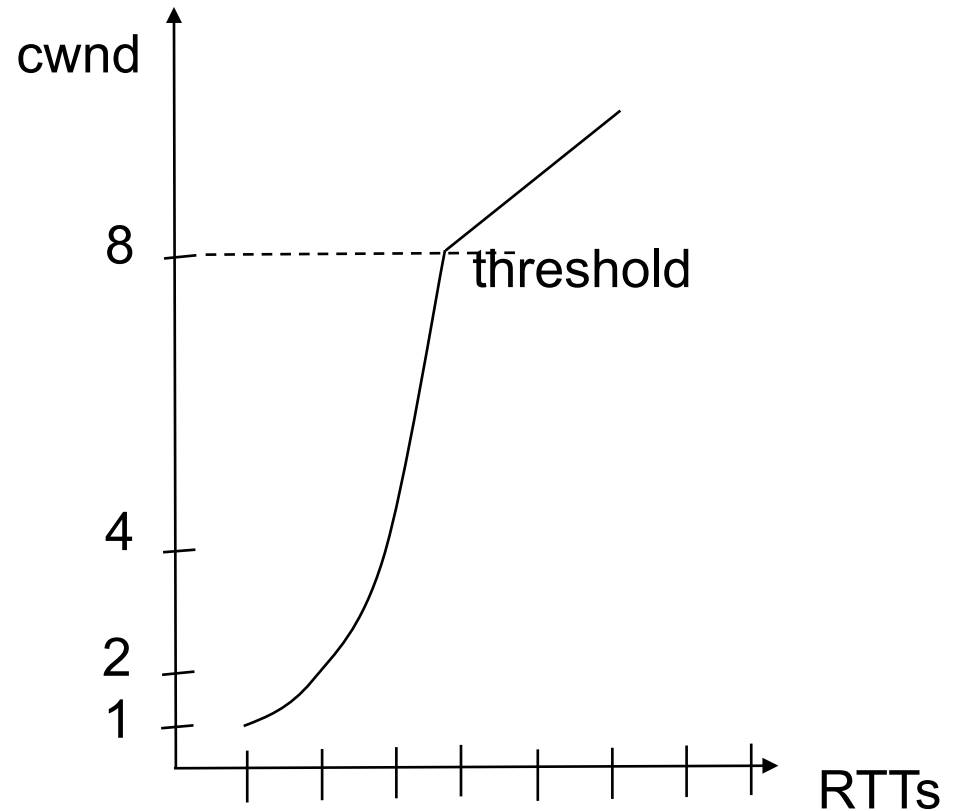
- **Slow start:** increase congestion window size by one segment upon receiving an ACK from receiver
 - initialized at ≤ 2 segments
 - congestion window increases exponentially



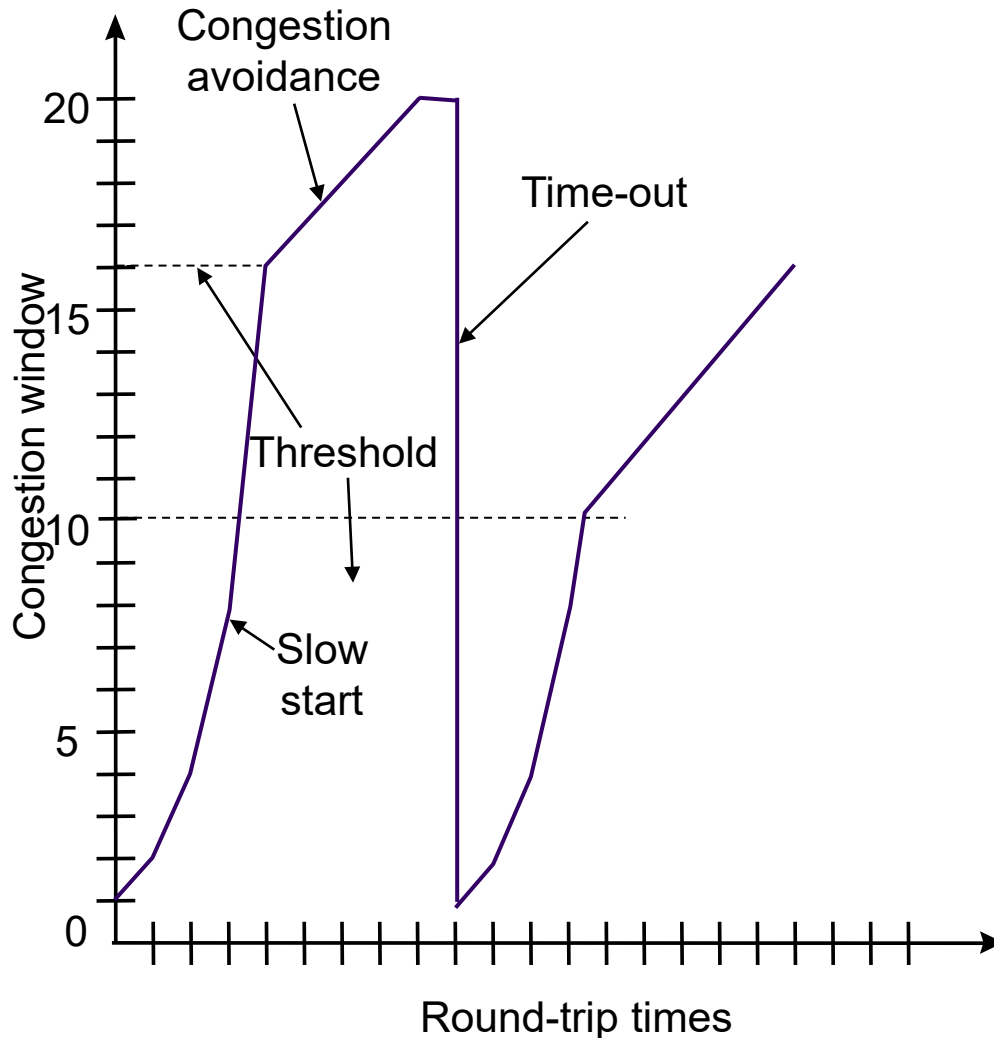
TCP Congestion Control: Congestion Avoidance



- Algorithm progressively sets a *congestion threshold*
 - When $\text{cwnd} > \text{threshold}$, slow down rate at which cwnd is increased
- Increase congestion window size by one segment per round-trip-time (RTT)
 - In one RTT, cwnd segments are sent, so total increase in cwnd is cwnd
 - cwnd grows linearly with time



TCP Congestion Control: Congestion Onset

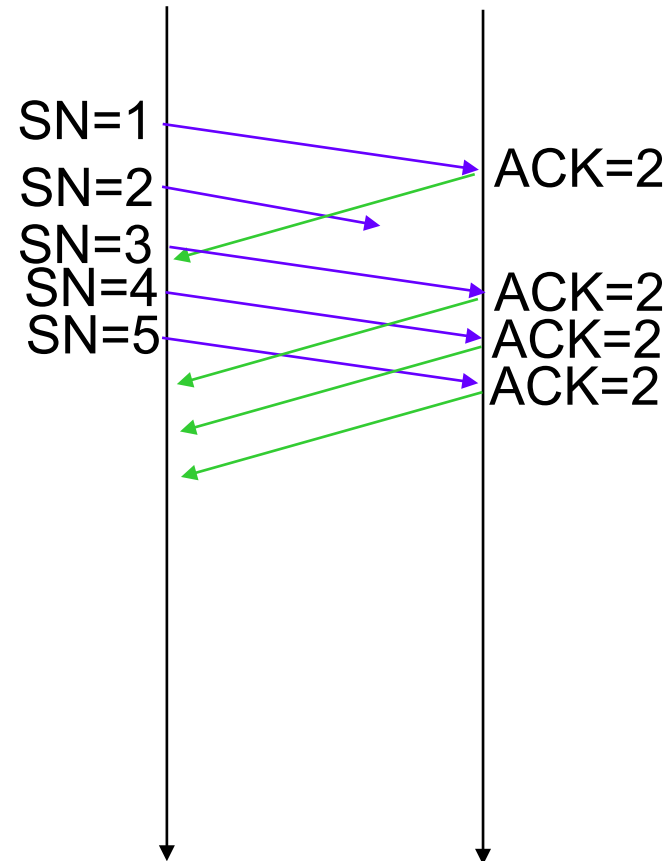


- Congestion is detected upon timeout or receipt of duplicate ACKs
- Assume current cwnd corresponds to available bandwidth
- Adjust congestion threshold = $\frac{1}{2} \times$ current cwnd
- Reset cwnd to 1
- Go back to slow-start



Fast Retransmit & Fast Recovery

- Congestion causes many segments to be dropped
- If only a single segment is dropped, then subsequent segments trigger duplicate ACKs before timeout
- Can avoid large decrease in cwnd as follows:
 - When three duplicate ACKs arrive, retransmit lost segment immediately
 - Reset cwnd to congestion threshold + 3 to account for the three segments that triggered duplicate ACKs
 - Remain in congestion avoidance phase
 - However if timeout expires, reset cwnd to 1
 - In absence of timeouts, cwnd will oscillate around optimal value



TCP Congestion Control: Fast Retransmit & Fast Recovery

